

SIMATS ENGINEERING

Department of Computer Science & Engineering

ASSESSMENT TOOL 2 – DEBUGGING & OPTIMISATION TASK

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO2	Assessment:	ASSESSMENT TOOL2- DEBUGGING & OPTIMISATION TASK
Weightage:	20%	Total Marks:	25
CO2 Statement:	<i>Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)</i>		
Student Name:	KAVALI BALAJI	Reg.No.:	192525402
Department:	AL&ML	Date:	1/8/2026

ANNEXURE

1. A program performing signed integer addition using 2's complement produces incorrect results when adding large values (e.g., +120 and +50 in 8-bit representation). Debug the issue by analyzing the binary computation and identifying whether overflow has occurred. Propose a solution to correctly detect and handle overflow in such cases.
2. A processor using a ripple carry adder shows significant delay in executing arithmetic operations in a real-time system. Debug the performance bottleneck by examining how carry propagates through each bit. Suggest an optimized solution using a carry look-ahead adder, and explain how it reduces delay.
3. A multiplication unit implementing Booth's algorithm gives incorrect results for certain negative operands. Debug the step-by-step execution of the algorithm, focusing on bit-pair checking, arithmetic shifts, and sign extension. Identify the possible source of error and suggest corrections to ensure accurate signed multiplication.
4. An embedded system using the restoring division algorithm experiences inefficiency due to repeated restoration steps, leading to increased execution time. Debug the algorithm's workflow and identify where unnecessary operations occur. Propose an optimized approach using the non-restoring division method, explaining how it improves performance.
5. A scientific application using IEEE 754 single precision floating-point representation produces inaccurate results when adding very small and very large numbers. Debug the issue by analyzing exponent alignment, normalization, and rounding errors. Suggest optimization techniques, such as using double precision or algorithmic adjust

Code of Conduct:

I K A V A L I B A L A J I (1 9 2 5 2 5 4 0 2) _ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:**MARKS ALLOCATION**

	Identification of Errors / Bugs (20%)	Debugging Approach & Analysis (20%)	Correctness of Debugged Solution (25%)	Optimization Techniques Applied (20%)	Testing and Documentation (15%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Identification of Errors / Bugs	20%	Accurately identifies all errors and issues with complete understanding of the problem	Identifies most errors with minor omissions	Identifies limited errors; misses key issues	Unable to identify errors or misunderstands the problem
Debugging Approach & Analysis	20%	Uses effective debugging techniques with clear analysis and root cause identification	Applies suitable debugging methods with minor gaps	Uses basic debugging methods with limited analysis	No systematic debugging approach followed
Correctness of Debugged Solution	25%	Provides completely corrected solution with accurate functionality and expected output	Provides working solution with minor errors	Partially correct solution with limited functionality	Incorrect solution or fails to resolve errors
Optimization Techniques Applied	20%	Applies efficient optimization methods to improve performance, memory usage and quality	Performs optimization with noticeable improvements	Limited optimization with minimal improvements	No optimization applied or inefficient implementation
Testing and Documentation	15%	Performs complete testing with proper validation and clear documentation	Provides adequate testing and documentation with minor issues	Limited testing and incomplete documentation	No proper testing or documentation provided

Q1. Signed Integer Addition Using 2's Complement

Problem:

A program performing signed integer addition using 2's complement produces incorrect results when adding +120 and +50 using 8-bit representation.

1. Error Identification

For an 8-bit signed 2's complement number, the representable range is -2^7 to 2^7-1 , i.e., -128 to $+127$. Therefore, $+170$ cannot be represented.

$$120 + 50 = 170$$

2. Binary Calculation

$$120 = 01111000_2$$

$$50 = 00110010_2$$

$$\begin{array}{r} 01111000 \\ + 00110010 \\ \hline 10101010 \end{array}$$

The binary result is 10101010_2 . Because the MSB is 1, this bit pattern represents a negative signed value.

$$10101010 \rightarrow \text{invert: } 01010101 \rightarrow \text{add 1: } 01010110_2 = 86, \text{ so } 10101010_2 = -86.$$

3. Overflow Identification

Both operands are positive, but the result has a sign bit of 1. Therefore, signed overflow has occurred. The mathematical result $+170$ is outside the 8-bit signed range.

Overflow = YES

4. Correct Debugged Solution

Overflow can be detected using the sign-bit rule: positive + positive producing a negative result indicates overflow; negative + negative producing a positive result also indicates overflow. Another hardware-level test is $V = \text{Cin(MSB)} \text{ XOR } \text{Cout(MSB)}$.

The program should set/report an overflow condition instead of treating 10101010 as the valid signed answer. If the application requires the value 170, a wider representation such as 16-bit signed arithmetic should be used.

5. Testing

Input	Mathematical Result	8-bit Result	Overflow
120 + 50	170	10101010	Yes
50 + 30	80	01010000	No
100 + 20	120	01111000	No
-100 + -50	-150	Out of range	Yes

Conclusion:

The error occurs because +170 cannot be represented in 8-bit signed 2's complement. Detecting the overflow flag and using a wider data type when required gives a correct and reliable solution.

Q2. Ripple Carry Adder and Carry Look-Ahead Adder

Problem:

A processor using a ripple carry adder experiences significant delay because carry propagates sequentially through each bit.

1. Error Identification

In a ripple carry adder, each full adder waits for the carry generated by the previous stage. For an 8-bit operation, a carry may propagate from bit 0 through bit 7 before the final result is stable.

2. Example Calculation

$$11111111_2 + 00000001_2 = 100000000_2$$

At bit 0, $1 + 1$ produces sum 0 and carry 1. That carry then propagates through the remaining stages. This sequential dependency is the performance bottleneck.

3. Debugging Analysis

For bit i , define the propagate and generate signals as:

$$P_i = A_i \text{ XOR } B_i$$

$$G_i = A_i B_i$$

The carry equation is $C_{i+1} = G_i + P_i C_i$.

4. Carry Look-Ahead Solution

A carry look-ahead adder calculates carry signals using propagate and generate terms rather than waiting for the previous stage.

$$C_1 = G_0 + P_0 C_0$$

$$C_2 = G_1 + P_1 G_0 + P_1 P_0 C_0$$

$$C_3 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0$$

Thus, multiple carry conditions can be evaluated in parallel, reducing the critical carry-propagation delay.

5. Optimization Comparison

Feature	Ripple Carry Adder	Carry Look-Ahead Adder
Carry generation	Sequential	Parallel/look-ahead
Delay	Higher	Lower
Hardware	Simple	More complex
Speed	Slower	Faster
Real-time suitability	Lower	Higher

6. Testing

For $255 + 1$: $11111111_2 + 00000001_2 = 100000000_2$. Both architectures produce the same arithmetic result, but the carry look-ahead design reduces the time required for carry computation.

Conclusion:

The bottleneck is sequential carry propagation. A carry look-ahead adder improves speed by computing carry signals from propagate and generate information in parallel.

Q3. Booth's Algorithm for Signed Multiplication

Problem:

A multiplication unit using Booth's algorithm gives incorrect results for negative operands.

1. Example

Calculate $(-5) \times (+3)$ using 4-bit 2's complement.

$+5 = 0101_2$, therefore $-5 = 1011_2$. Multiplier $+3 = 0011_2$.

Initial values: $M = 1011$, $-M = 0101$, $Q = 0011$, $A = 0000$, $Q-1 = 0$.

2. Booth Decision Rules

$Q_0 Q_{-1}$	Operation
00	No operation
01	$A = A + M$
10	$A = A - M$
11	No operation

After the required addition/subtraction, an arithmetic right shift is performed on the combined A, Q, $Q-1$ registers. The process is repeated n times; for 4-bit operands, 4 iterations are required.

3. Debugging Points

Bit-pair checking: The algorithm must inspect the pair (Q_0, Q_{-1}) . Checking only Q_0 can produce incorrect operations.

Arithmetic shift: A signed arithmetic right shift preserves the sign bit. For example, 1010 shifts to 1101, not 0101.

Sign extension: Negative intermediate values must retain the sign bit when shifted. Failure to sign-extend changes the numerical value.

Initialization: $Q-1$ must start at 0, and the accumulator must be initialized correctly.

4. Expected Result

$$(-5) \times 3 = -15.$$

$$15 = 00001111_2; \text{invert} \rightarrow 11110000_2; \text{add } 1 \rightarrow 11110001_2.$$

$$\text{Therefore, } -15 = 11110001_2.$$

5. Possible Sources of Error

1. Incorrect Q_0, Q_{-1} pair checking.
2. Logical instead of arithmetic right shift.
3. Failure to sign-extend negative operands.
4. Incorrect subtraction operation.
5. Incorrect initialization of $Q-1$.
6. Incorrect number of iterations.

6. Corrected Algorithm

1. Represent operands in 2's complement.
2. Initialize $A = 0$ and $Q-1 = 0$.
3. Inspect Q_0 and Q_{-1} .
4. Perform $A + M$ or $A - M$ when required.

5. Arithmetic-right-shift A,Q,Q-1.
6. Repeat exactly n times.
7. Combine A and Q for the final product.

7. Testing

Multiplication	Expected Result
5×3	15
-5×3	-15
5×-3	-15
-5×-3	15

Conclusion

Correct bit-pair checking, arithmetic shifting, sign extension, initialization, and iteration count are essential for accurate signed Booth multiplication.

Q4. Restoring Division and Non-Restoring Division

Problem:

Restoring division is inefficient because an unsuccessful subtraction is followed by a restoration operation.

1. Restoring Division Workflow

The restoring algorithm shifts the partial remainder and quotient, subtracts the divisor, checks the sign, and restores the previous remainder when the subtraction produces a negative result.

2. Example

$13 \div 3 = 4$ remainder 1.

$13 = 1101_2$ and $3 = 0011_2$.

Therefore, quotient $Q = 0100_2 = 4$ and remainder $R = 0001_2 = 1$.

3. Inefficiency

When $R - M < 0$, the algorithm restores the partial remainder by adding the divisor back: $R = (R - M) + M$. This extra addition increases execution time.

4. Non-Restoring Division

Non-restoring division avoids immediate restoration. If the partial remainder is positive, the divisor is subtracted; if it is negative, the divisor is added in the next step.

If $R \geq 0$: $R = 2R - M$

If $R < 0$: $R = 2R + M$

The quotient bit is determined from the sign of the updated partial remainder.

5. Comparison

Operation	Restoring Division	Non-Restoring Division
Subtraction	Yes	Yes
Immediate restoration	Required when negative	Avoided
Extra operation	Frequent restore addition	Reduced
Speed	Lower	Higher
Efficiency	Lower	Better

6. Testing

For $13 \div 3$, the expected result is $Q = 4$ and $R = 1$.

Verification: Dividend = (Divisor $\times$ Quotient) + Remainder = $(3 \times 4) + 1 = 13$. Hence the result is correct.

Conclusion:

Non-restoring division improves performance by avoiding immediate restoration and selecting addition or subtraction based on the sign of the partial remainder.

Q5. IEEE 754 Single-Precision Floating-Point Addition

Problem:

A scientific application produces inaccurate results when adding very small and very large numbers.

1. IEEE 754 Single Precision

Single precision contains 1 sign bit, 8 exponent bits, and 23 fraction (mantissa) bits, for a total of 32 bits. The exponent bias is 127.

2. Example

Consider $1.0 + 2^{-25}$.

$$1.0 = 1.00000000000000000000000 \times 2^0$$

$$2^{-25} = 1.00000000000000000000000 \times 2^{-25}$$

3. Exponent Alignment

Before addition, the exponents must be made equal. The smaller number's significand is shifted right by 25 positions so that it aligns with exponent 0. Because single precision has limited fraction precision, significant bits of the smaller number can be lost.

Consequently, in single precision, $1.0 + 2^{-25}$ can round to approximately 1.0, even though the exact mathematical result is 1.0000000298023224.

4. Normalization and Rounding

After mantissa addition, the result is normalized to the form $1.xxxxx \times 2^E$. Any bits beyond the available precision are handled by rounding. When the operands differ greatly in magnitude, the smaller operand may have little or no effect on the stored result.

5. Optimization Techniques

Use double precision when greater numerical accuracy is required. Double precision has 1 sign bit, 11 exponent bits, and 52 fraction bits, giving substantially more precision than single precision.

Algorithmic improvements include grouping values with similar magnitudes, using compensated summation for long sums, and reducing unnecessary intermediate rounding.

6. Testing

Issue	Cause	Solution
Exponent mismatch	Different exponent values	Align exponents
Mantissa loss	Limited fraction bits	Use higher precision
Rounding error	Finite precision	Use stable numerical methods
Small value disappears	Large magnitude difference	Double precision/compensated summation

Conclusion:

The inaccuracy is caused by exponent alignment, limited significand precision, normalization, and rounding. Double precision and numerically stable algorithms improve accuracy.